/Interfaces
E
For each/

Flora Belas

/For each/

- É uma forma de iterar elementos de uma array ou collection de forma mais simples, mas menos flexível.

/For each/

- Forma **curta** de percorrer quando você só quer **os valores**
- Lê-se: *para cada nome dentro de nomes*
- O **:** se lê como **"dentro de"**
- Não precisa escrever início, condição nem passo.
- O tipo antes do nome é **o tipo do que está dentro**

/Estrutura /

- for(String nome : nomes)
- | | | |
- Tipo da array apelido in Seu array

Lê-se: **Para cada nome dentro de nomes...**

/Exemplo/

```
ArrayList<String> nomes = new ArrayList<>();
nomes.add("Ana");
nomes.add("Bia");

for (int i = 0; i < nomes.size(); i++) {
    System.out.println(nomes.get(i));
}

for (String nome : nomes) {
    System.out.println(nome);
}
```

/Limitações/

- **Você não sabe em que posição está** (sem acesso ao índice)**.** Se precisar imprimir "Nota 1", "Nota 2", use o for normal com o i.
- **Não dá pra alterar o array original.** O for-each cria uma "cópia temporária" do valor do item a cada repetição. Se você tentar mudar o valor da variável de dentro do laço, o array original **não será alterado**. Para modificar os valores originais, você precisa do for tradicional usando o índice.
- Dá erro se você tentar remover itens do array durante ele.
- Só anda para frente e de um em um.

/Limitações/

Por isso o **for-each** é excelente e o mais recomendado quando você só precisa **ler e usar** os dados que já estão guardados, **sem alterar a estrutura da lista**.
Use para olhar, comparar, exibir ou somar itens da lista.
Interfaces podem ser tipos de listas, parâmetro de método, retorno de método....

/Interfaces/

- Uma **interface em Java** funciona como um contrato ou uma lista de regras. Ela diz **o que** uma classe deve fazer, mas não **como** ela faz isso.

/Interfaces/

No Java, a interface define **métodos sem corpo** (sem código dentro das chaves {}). Qualquer classe que "assinar" (implementar) essa interface é **obrigada** a escrever o código de todos esses métodos. Nada roda na interface, é só uma promessa.
Na interface tudo é publico. **Os métodos a serem implementados nas classes são obrigatoriamente públicos.**

/Estrutura/

```
public interface minhaInterface{

    void meuMetodo();
    void outroMetodo();
}
```

```
public class minhaClasse implements minhaInterface{

        public void meuMetodo(){

        }
        public void outroMetodo(){

        }
}
```

/implements/

- **implements** é assinar o contrato. Toda vez que sua classe implementar uma interface, ela vai precisar implementar TODOS os métodos que a interface pede.
- É a palavra chave usada por uma classe para seguir o contrato.
- Classes podem implementar múltiplas interfaces.

/@Override/

- **Override** é uma annotation (uma anotação com instruções para o compilador ler depois)
- É a palavra chave usada por uma classe para seguir o contrato.
- **Não é código que** roda. É um recado pro compilador. “Lê-se: *"compilador, esse método aqui é pra estar cumprindo um contrato, confere pra mim“*
- Fica em cima de cada método implementado das classes que assumiram o contrato.

/@Override/

- Com o **Override** o Java exige que a implementação esteja correta.

/Uma origem, várias formas/

Uma Interface funciona como um **tipo universal**. Se dez classes diferentes assinarem a mesma interface, você pode usar o tipo da interface para controlar e instanciar qualquer uma delas.

/Uma origem, várias formas/

Imagine a interface Animal, com as classes assinando ela sendo Cachorro, Gato, Papagaio.

Um único tipo: Você pode criar uma variável com o tipo da Interface.

```
Animal animal
```

Múltiplas identidades: Essa mesma variável pode instanciar e armazenar **classes totalmente diferentes** (desde que assinem a interface).

```
Animal cachorro = new Cachorro();
Animal gato = new Gato();
Animal papagaio = new Papagaio();
```